

动态规划简单题选讲 2

Purslane

云斗学院 · 省选培训 · 第一阶段

2025.12.30

[AGC036F] Square Constraints

题目大意

给定正整数 N 和模数 M ，求有多少个 $(0, 1, \dots, 2N - 1)$ 的排列 $(P_0, P_1, \dots, P_{2N-1})$ ，满足：

$$N^2 \leq i^2 + P_i^2 \leq 4N^2。$$

对 M 取模。

$$1 \leq N \leq 250, 1 \leq M \leq 10^9, 4s。$$

题解

将下标分为两类： $0 \leq i < N$ 的为一类， $N \leq i < 2N$ 的为另一类。

如果只考虑 $i^2 + P_i^2 \leq 4N^2$ ，那么 P_i 有上限 Q_i 。

将 Q 从小到大排序， $\prod_{i=0}^{2N-1} (Q_i + 1 - i)$ 就是答案。

考虑容斥，钦定部分一类数满足 $i^2 + P_i^2 < N^2$ ，假设钦定了 k 个给 $(-1)^k$ 的容斥系数即可。

假设我们钦定了 k 个位置，如何计算每个数的排名？

假设一类点的限制是 $U_{1,i} < P_i \leq U_{2,i}$ ，二类点的限制是 $P_i \leq U_{1,i}$ 。

那么按照 $U_{1,i}$ 排序，我们将部分一类点提升。只需要记录当前提升了多少个数，我们就很容易知道每个数是否提升的排名。

复杂度 $O(n^3)$ ，因为你需要提前枚举 k 。

P12251 [科大国创杯初中组 2025] 抽卡

题目大意

有 m 种卡牌，每种卡牌有无限张，第 i 种卡牌价值为 i 。
我们将进行 n 次抽卡：给定序列 $a_1, 2, \dots, n$ ，第 i 次抽卡会从前 $\sum_{j=1}^i a_j$ 种卡牌里随机选一种，在这种卡牌里面抽取两张放到牌堆的最后。

确定牌堆之后，A 和 B 进行博弈。A 每次随便取走一张卡牌，B 只会抽取最左边的卡牌。

问在两者都很聪明的情况下，A 的期望收益为多少。

$1 \leq n \leq 500$, $1 \leq m = \sum a_i \leq 10^9$, $0 \leq a_i \leq m$, 2s。

问题转化

假设告诉你了牌堆，如何确定 A 的最优方案？容易发现，B 是个波特，所以只需要确定 A 可以选的卡牌集合就行。

显然，A 只能在前 $2k$ 个卡牌里面选最多 k 张。然而，我们既不能从前往后维护堆，也不能直接贪。因此，我们可以改写为：后 $2k$ 张卡牌里面要选至少 k 张。

这样 A 的策略就是：维护一个堆，每次从卡牌里面加两张牌到堆里，然后从堆中取最大元素。

这是一个很固定的策略，可以通过 DP 套 DP 的方法进行维护。直接维护堆是不现实的，但是如果你把 x 拆成： $\sum_{v \geq 1} [x \geq v]$ ，也就是通过二分转成 01 问题。

容易发现，每个 01 通过堆得出的策略，和直接用堆得出的策略，是本质相同的，因此将 $O(V)$ 个问题直接累加起来就是原问题。

题解

转成 01 之后，容易发现只需要维护堆里面有多少个 1。使用 1 的总个数，减去堆里剩余 1 的个数，就是选出的 1 的个数。

仍然拆贡献，拆完之后发现对于 v ，只需要 $dp_{i,j}$ 表示考虑了所有 $\geq i$ 的位置，堆里面有 j 个 1 的概率。

那么， $dp_{i,j}$ 以 $\max\{0, kv + b\}$ 的概率转移到 $dp_{i-1,j}$ ，以 $1 - \dots$ 的概率转移到 $dp_{i-1, \max(0, j-1)}$ 。

容易发现，我们可以把 v 分为 n 段，每一段都可以把概率改写为关于 v 的一次函数。

因此，最终 $\sum_{i=1}^n idp_{1,i}$ 是关于 v 的不超过 n 次函数 $f(v)$ 。

答案计算

相当于：给你 v_1 和 v_2 ，计算 $\sum_{v=v_1}^{v_2} f(v)$ ， f 是一个不超过 n 次函数。

思路 1： $\sum_{i=1}^v f(i)$ 是关于 v 的至多 $O(n)$ 次函数，带入 $O(n)$ 个值就可以直接插值弄出来；

思路 2：由于每次只会乘上一次函数，所以暴力维护多项式即可，然后用插值或者斯特林公式计算等幂和。

思路 3：等幂和的计算是很困难的，但是如果能把它写成 $\sum_{i=0}^{n+2} a_i \binom{v}{i}$ 的形式，那么求和是容易的。因此直接维护下降幂多项式即可。

CF2062F Traveling Salescat

题目大意

有 n 个城市，每个城市有两个参数 a_i 和 b_i 。

城市 (i, j) 之间的道路权值为 $\max(a_i + b_j, a_j + b_i)$ 。

对于每个 $2 \leq k \leq n$ ，求包含 k 个城市的简单路径的最短路径长度。

$1 \leq n \leq 3 \times 10^3$, $0 \leq a_i, b_i \leq 10^9$, 2s。

确定最优结构——放缩法

这种“在比较复杂”的结构中找最优，一般来说是：先通过简单的分析（调整，放缩）等，得到最优解的某个形态，然后再对着这个形态做 DP。

容易发现，权值关于 i, j 过于有联系，我们不好去分析最优解的结构。

令 $t_i = b_i - a_i$ ，那么 (i, j) 的权值就是 $a_i + a_j + \max(t_i, t_j)$ 。

固定首尾之后（注意首尾对 a 的贡献不一样），我们相当于：固定 t_0 和 t_{n+1} 。通过适当排列使得 $\sum_{i=0}^n \max(t_i, t_{i+1})$ 最小。

而 $\max(a, b) = \frac{a+b+|b-a|}{2}$ ，因此你最小化 $\sum_{i=0}^n |t_i - t_{i+1}|$ 即可。

不妨设 $t_0 \leq t_{n+1}$ 。

你取 $t_{\min}$ 表示 $t_{1,2,\dots,n}$ 的最小值， $t_{\max}$ 表示最大值，发现这个最小值有下界 $|t_0 - t_{\min}| + |t_{\min} - t_{\max}| + |t_{n+1} - t_{\max}|$ 。

题解

这个下界在 $t_1 \leq t_2 \leq \dots \leq t_n$ 时取到。这样，我们弄明白了最优结构：除了首尾，中间数按照 t 从小到大排列。
那么按照 t 从小到大的顺序插入最终的序列，我们用以下两个量来描述当前序列：

- (1) 中间部分的长度；
- (2) 0（表示首尾都没放）；1（只有首放了）；2（首尾都放了）。

容易进行转移，复杂度 $O(n^2)$ 。

P12452 [INOI Team Selection 2021] Color Colony

题目大意

给你一棵树 T 。求出最大的 C ，使得你可以将 T 的边染上 C 种颜色（每种颜色都需要出现），且不存在长度为 k 的路径，使得 k 条边两两不同色。

$1 \leq k \leq 30$, $1 \leq k \leq n \leq 10^5$, 1s。

确定最优结构——调整法

先考虑随便给出一个染色。

如果颜色 r 形成了若干个连通块，取出这些连通块的虚树。

考虑虚树上的一个叶子结点，那么这个叶子节点走向其他连通块，第一个其他颜色的边 r' 是确定的。

容易知道把这个连通块所有颜色换成 r' 更优。因此，最优结构告诉我们：最优解一定所有颜色形成一个连通块。

题解

考虑进行树上 DP。如果我们确定了 u 子树内所有的边，看看我们需要知道什么。

我们需要考虑 $u \rightarrow s$ （其中 s 是 u 的某个儿子）的不经过重复颜色路径中，最长与次长（注意次长要求 $u \rightarrow son$ 的第一条边和最长不一样）。

所以对于每个子树，只需要维护最长和次长两个就行了（因为你要考虑 $u \rightarrow fa$ 的颜色。如果是和最长相同，新的最长是原次长加一；否则，颜色数加一，新的最长是原最长加一）。

考虑 DP 合并若干状态。发现我们不太容易合并两个状态（否则，至少得 $O(k^3)$ 这种），但是我们可以暴力去枚举最终的最长和次长。

错解不优：直接钦定最后结果

不妨直接钦定最后最长为 $len1$ ，次长为 $len2$ 。则有一个颜色最长 $\leq len1$ ，其他都 $\leq len2$ （错解不优）。

注意必须有一个数选最大值，可以开一个 01 记录一下。

这样使用前缀和优化就可以做到 $O(k^2)$ ：

- (1) 新加入一个子树的时候，如果 $\leq len2$ 直接认为是不同于最长的颜色，加上 $dp_{i,len}$ ；
- (2) 否则，我们认为这个就是最长的，加上 $dp_{i,len} - 1$ 。

总复杂度 $O(nk^2)$ 。

P14344 [JOISC 2019] 两道料理 / Two Dishes

题目大意

你要做两种料理。

两种料理分别有 n 和 m 步，你需要按照顺序完成每一种料理，并且你不能休息。

完成第一种料理的第 i 步需要 A_i 的时间，完成第二种料理的第 i 步需要 B_i 的时间。

有若干次评分，每种评分为以下两种：

- (1) 如果在 S_i 时刻前完成了第一种料理的第 i 步将获得 P_i 分；
- (2) 如果在 T_i 时刻前完成了第二种料理的第 j 步将获得 Q_i 分。

问你最多能获得多少分。

$$1 \leq n, m \leq 10^6, \quad 1 \leq A_i, B_i \leq 10^9, \quad 1 \leq S_i, T_i \leq 2 \times 10^{15}, \\ -10^9 \leq P_i, Q_i \leq 10^9, \quad 5s.$$

题解

这些限制比较诡异，我们很容易将其转化为这种形式：如果完成第一道料理的第 X_i 步的时候另一道料理已经完成了 Y_i 步，那么获得 K_i 的得分。

设 $dp_{x,y}$ 表示：完成第 x 道料理的时候，第 y 道料理完成了 j 步的最大代价。

考虑 $dp_{x,y}$ 如何从 $dp_{x-1,y}$ 演变而来：

- (1) 首先，对 $dp_{x-1,*}$ 求前缀 $\max$ ，表示在第 x 步之前完成了若干步；
- (2) 然后，将 dp_{x-1} 拷贝到 dp_x 上来，再做若干次后缀加。

题解

由于有前缀 $\max$ 这种操作，无法使用常见数据结构直接维护 dp 的值。

然而，如果能维护 dp 数组的差分值，那么取前缀 $\max$ 就是很容易的事情：使用 set 进行差分数组的推平。

而后缀加对差分数组本身就很友好——相当于在差分数组上单点加。

因此容易将本题做到 $O(n \log n)$ 。

CF1603D Artistic Partition

题目的大意

定义 $c(l, r) = \sum_{i=l}^r \sum_{j=i}^r [\gcd(i, j) \geq l]$ 。

定义 $f(n, k)$ 为：将 $[1, n]$ 划分为 k 段

$(0 = x_1 < x_2 < \dots < x_{k+1} = n)$ ， $\sum_{i=1}^k c(x_i + 1, x_{i+1})$ 的最小值。

有 q 次询问，每次个给出 (n, k) ，请你求出 $f(n, k)$ 。

$1 \leq q \leq 3 \times 10^5$ ， $1 \leq k \leq n \leq 10^5$ ，3s。

题解

显然 $i = j$ 都有贡献，我们不看——直接处理

$$c(l, r) = \sum_{i=l}^r \sum_{j=i+1}^r [\gcd(i, j) \geq l].$$

当 $r < 2l$ 的时候， $c(l, r) = 0$ 。因此我们一定可以将 n 划分为 $O(\log n)$ 段，代价为 0。只需要考虑 $k = O(\log n)$ 的情况。

设 $dp_{i,j}$ 表示将 i 分为 j 段，从 $dp_{*,j-1}$ 转移到 $dp_{*,j}$ 。如果决策点 $i_1 < i_2$ 且对某个 i_0 ， i_2 更优，则对所有的 $i \geq i_0$ 都是 i_2 更优。因此具有决策单调性，对于这种离线的问题可以整体二分（或者分治法）。

众所周知，整体二分对于两个端点的变化（就是我们需要直接计算 $c(l, r)$ ）是 $O(n \log n)$ 的，因此只需要知道：如果我们算出了 $c(l, r)$ ，如何计算 $c(l \pm 1, r)$ 和 $c(l, r \pm 1)$ 。

如何处理两端点的扩展

- (1) 移动 l : 问区间中有多少个东西的 $\gcd$ 恰好为 $l-1$ 。这个是简单数论问题，直接预处理即可。
- (2) 移动 r : 问 r 和 $[l, r-1]$ 中多少个 $\gcd$ 要 $\geq l$ 。枚举 r 的因数 d ，有 $\varphi(r/d)$ 个数和他的 $\gcd$ 为 d 。那么相当于对 r 的因子序列求前缀和，可以做到 $O(\log n)$ 。

因此，我们可以 $O(n \log^3 n)$ 解决本题。

第二种想法：记 $f(n) = \sum_{i=1}^n \sum_{j=i+1}^n [\gcd(i, j) = 1]$ 。

容易发现 $c(l, r) = \sum_{i=l}^r f(\lfloor r/i \rfloor)$ 。事实证明，直接暴力算这个就可以过（使用整除分块）。

而注意到整除分块跳一步之后只有 $O(\sqrt{r})$ 种情况，把他们都预处理出来，就可以 $O(n\sqrt{n})$ 预处理、 $O(1)$ 计算 c ，复杂度 $O(n\sqrt{n} + n \log^2 n)$ 。

处理决策单调性问题的几种方法

首先，你要看出来问题具有决策单调性。一般通过打表、四边形不等式分析（参考 OI-Wiki，内容很完善），以及感性理解法。感性理解意思是：你只需要证明，如果某一个时刻 $i_2 > i_1$ 更优，则右端点往后扫的时候 i_2 增量会更小，那么有决策单调性）。这种决策单调性 DP 分为：离线问题（例如本题）和半在线问题。离线问题一般使用分治法处理，其优点是需要的代价函数两个端点的移动是 $O(n \log n)$ 的（这叫做滑动的内存访问）。而大家对于半在线的问题一般使用二分栈，其缺点是：我们对代价函数的访问是随机的，对于类似本题的问题是不好处理的。半在线是有方法做到内存连续的，可以参考 [这篇文章](#)。好像还有一些其他算法（比如 SMAWK），可以参考 [这篇文章](#)。

[AGC040E] Prefix Suffix Addition

题目大意

给定非负整数序列 a 。

你可以进行以下两种操作：

- (1) 选择长度 k 以及 $0 \leq c_1 \leq \dots \leq c_k$ ，让 $a_i \leftarrow a_i - c_i$ ；
- (2) 选择长度 k 以及 $0 \leq c_n \leq \dots \leq c_{n-k+1}$ ，让 $a_i \leftarrow a_i - c_i$ 。

问最少要进行多少次操作。

$1 \leq n \leq 2 \times 10^5$, $1 \leq A_i \leq 10^9$ 。

基础 DP

假设只有第一种方法，答案就是 $\sum_{i=0}^n [a_i > a_{i+1}]$ 。

因此，我们可以考虑给每个 $0 \leq i \leq n+1$ 分配一个 $0 \leq c_i \leq a_i$ ($a_0 = a_{n+1} = 0$)，那么：

$\sum_{i=0}^n [c_i > c_{i+1}] + [a_i - c_i < a_{i+1} - c_{i+1}]$ ，我们将其最小化。

因此可以设 $dp_{i,j}$ 为，考虑了前 i 个数的贡献，并且 $c_i = j$ 的最小答案。

这时候如果你打表，容易发现以下两个结论：

- (1) $dp_{i,*}$ 单调不增，因为容易看到固定 c_{i-1} 的时候， c_i 越大越好；
- (2) dp_i 的极差最多为 2，因为每次最多新增两个艾弗森括号。

题解

因此，我们可以不去记录 dp 数组，而是记录 dp 数组的连续段（这里最多只有三段，记录几个分界点即可）。

通过复杂的分类讨论是可以直接退出分界点之间的关系的，不过我们做题的原则是——如果能让代码帮我们分类讨论，我们自己就不分类讨论。

注意 $dp_{i,j} = \min_{0 \leq k \leq a_{i-1}} dp_{i-1,k} + [k > j] + [a_{i-1} - k < a_i - j]$ 。

注意我们的 k 只有 $O(1)$ 种取值： a_{i-1} ， j ， $a_{i-1} - a_i + j$ 。

只有 k 是断点， j 才可能是断点，因此可能得断点只有 $O(1)$ 个，暴力计算它们的值即可。

这种方法本质上是：如果 dp 数组很有规律，我们可以交换值域和定义域。

[AGC049E] Increment Decrement

题目大意

对于序列 a ，定义 $f(a)$ 为用以下两种操作把 a 变为 0 的最小代价：

- (1) 选择 $1 \leq i \leq n$ ， $a_i \leftarrow a_i \pm 1$ ，花费代价 1；
- (2) 选择 $1 \leq l \leq r \leq n$ 和 $x \in \{-1, 1\}$ ，将 $a_i \leftarrow a_i + x$ ($l \leq i \leq r$)，花费代价 C 。

给定 C 和 K ，你有 n 个长度为 K 的序列 $B_{1,2,\dots,n}$ 。

对于每个 $1 \leq i \leq n$ ， a_i 从 B_i 里面选。对于这 K^n 种方法，求 $f(a)$ 的和，对 $10^9 + 7$ 取模。

$1 \leq n, k \leq 50$ ， $1 \leq B_{i,j} \leq 10^9$ ， $1 \leq C \leq n$ 。

如何计算 f

我们的想法是：先用第一种操作把 a 修理一下，然后只需要进行 $\sum_{i=1}^{n+1} \max\{0, a_i - a_{i-1}\}$ 次二操作。

设 $dp_{i,j}$ 表示：我们把 a_i 弄成 j 的最小代价。

那么 $dp_{i,j} = |a_i - j| + \min_k \{dp_{i-1,k} + C \max\{0, j - k\}\}$ 。

最后 $dp_{0,0}$ 就是答案。这个式子显然可以使用 Slope Trick 优化：

记 $f_{i,j} = \min_k \{dp_{i-1,k} + C \max\{0, j - k\}\}$ ，这个东西就是：在 dp_{i-1} 的基础上，将斜率 < 0 的一部分推平，斜率 $> C$ 的一部分弄成斜率为 C 。

题解

不妨设初始状态下 $dp_{0,x} = \max\{0, cx\}$, 那么斜率分界点有 c 个。每次把 $f(x) = |a_i - x|$ 加入影响, 也就是插入两个 a_i 作为斜率分界点。

这时 dp_0 增加 a_i 。然后考虑推平, 也就是删掉最大的斜率分界点和最小的斜率分界点, 并且把 dp_0 减去最小的斜率分界点。容易发现, 这是一个类似堆的结构。那么依然使用 01 trick, 做 $O(nk)$ 次, 每次只需要记录堆里面有多少个 1 即可。

显然 $dp_{n+1,0}$ 就是答案, 容易做到 $O(Cn^2k)$ 。